

高级语言程序设计 实验报告

南开大学人工智能学院

姓名：胡宗义

学号：2510844

班级：0572

2026 年 5 月 3 日

Contents

1	作业题目	3
2	开发工具	3
3	作业要求	3
4	系统架构与设计思路	3
4.1	游戏思路	3
4.2	整体架构与模块划分	3
4.3	核心类的设计	4
4.3.1	Player	4
4.3.2	CameraController	4
5	关于 AI 工具使用的说明	4
5.1	GDExtension 相关问题的解决	4
5.2	引擎集成的技术咨询	4
5.3	核心代码实现的参考	5

1 作业题目

基于 Godot 引擎与 C++ GDExtension 的 2D 双人格斗游戏设计与实现

2 开发工具

1. 编程语言：C++
2. 游戏引擎：Godot4.6
3. 图形化框架：Godot-CPP (GDExtension)
4. 集成开发环境：Visual Studio Code
5. AI 辅助工具：腾讯元宝 (deepseek)

3 作业要求

自选题目，使用 C++ 语言完成一个图形化的小程序。图形化平台不限，可以是 MFC、QT 等任何 C++ 图形化平台 (不限制)。程序内容主题不限，看重创新、创意。

4 系统架构与设计思路

4.1 游戏思路

我首先将游戏类型铆钉为 2D 横板动作游戏，随后为了有所创新，我决定尝试实现双人“联机”功能。故其整体思路为：实现一个 Player 类描述玩家的属性与能力，在场景中创建两个 Player 类并实现独立控制，最后优化镜头以便我们能同时看见两个 Player。

4.2 整体架构与模块划分

由于原生的 Godot 引擎仅支持 GDscript 语言，我采用 C++GDExtension 进行开发，这样可以将 C++ 语言编写的脚本作为引擎的扩展脚本，从而既使用 C++ 语言要求，又能够享受到引擎的便利。项目核心主要划分为两个核心的 C++ 模块，通过 Godot 的节点树和信号机制进行通信与协同。

- **Player 模块**：作为游戏的绝对核心，继承自 Godot 中的 ‘CharacterBody2D’ 类。它不是一个单一功能模块，而是一个集成了物理运动、战斗系统、状态机、动画、碰撞体积和 UI 控制的复合实体。
- **CameraController 模块**：继承自 ‘Camera2D’，职责单一，专门负责计算并更新摄像机位置，以实现对两位玩家的智能跟随。
- **Godot 引擎运行时**：提供场景树、渲染循环、物理引擎、输入系统等基础设施。C++ 模块通过继承 Godot 内置类并重写虚函数嵌入引擎主循环。
- **通信机制**：模块间通过 Godot 的节点路径引用和信号-槽机制进行通信。

数据流：用户输入由 Godot 引擎捕获并传递给相应 ‘Player’ 实例的 ‘handle_input’ 方法；‘Player’ 在 ‘_physics_process’ 中更新自身所有状态（位置、血量、动画）及子 UI 节点；‘CameraController’ 在同一循环中读取两个 ‘Player’ 的位置，并独立更新摄像机坐标。

4.3 核心类的设计

4.3.1 Player

设计思路与功能

- 物理实体与控制器：继承自 ‘CharacterBody2D’ 类，它直接参与物理模拟，处理移动、跳跃、重力、碰撞。
- 战斗系统：在类内部实现了攻击检测、受击处理、伤害计算、击退逻辑和无敌帧。
- 状态机：通过一系列布尔标志（比如 ‘is_attacking’，‘is_rolling’，‘is_crouching’ 等）和枚举类型管理数十种状态及其转换，逻辑集中在 ‘update_state()’ 和 ‘_physics_process()’ 中。
- 动画与资源管理器：直接持有并控制 ‘AnimatedSprite2D’ 子节点，根据状态播放对应动画。同时管理 ‘AttackArea’ 和 ‘HurtBox’ 等子节点的引用与状态。
- UI 控制器：在 ‘initialize_ui()’ 和 ‘update_ui()’ 中，直接获取并操作其子节点树下的 ‘Label’（名称）和 ‘TextureProgressBar’（血条），将游戏逻辑状态实时反映到 UI 上。

最终，我实现了一个具有碰撞体积、血量系统、玩家标志、无敌帧、能够根据状态的不同播出不同动画的“玩家”。

4.3.2 CameraController

设计思路 每帧计算场景中两个指定 ‘Player’ 节点的中心点，并将摄像机移动至该位置。它不关心游戏规则，也不管理任何其他状态。

5 关于 AI 工具使用的说明

在本项目的开发过程中，我主要在以下几个技术环节使用了 AI 工具进行辅助：

5.1 GDExtension 相关问题的解决

- 编译错误排查：在编写 CameraController 类与 Player 类时，遇到了多次编译错误，AI 工具协助分析了错误信息并提供了可能的解决方案。
- 运行时问题诊断：当 GDExtension 扩展库加载失败或游戏启动异常时，AI 工具提供了系统的调试思路，包括添加调试输出、检查节点引用等方法。

5.2 引擎集成的技术咨询

- 信号与属性系统：在将 C++ 类绑定到 Godot 引擎时，对 _bind_methods() 的正确用法、属性暴露方式等进行了咨询。
- 节点查找策略：在实现 CameraController 的玩家节点查找逻辑时，我对于这种扩展与引擎的交互不太熟悉，于是就相关部分查询了 AI。

5.3 核心代码实现的参考

- bug 修复：我在实现诸多功能时，常常出现报错，当自行修改无果后，会求助于 AI。
- 代码结构优化：在 Player 类的状态机实现中，由于状态与判断条件繁多，于是我最终使用 AI 辅助简化了代码，优化了判断逻辑。